

Table of contents

C Program structure and examples.....	3
Basic Hello World Program.....	3
Program with User Input.....	3
Program with Formatted Output.....	4
GCC Compilation Process.....	5
C Compilation Stage Examples.....	5
Stage 1: Preprocessing.....	5
Stage 2 : Compilation to Assembly.....	5
Stage 3 : Assembly to Object File.....	6
Stage 4 : Linking.....	6
Return Values and Exit Status.....	6
Understanding Return Values.....	6
Checking Exit Status.....	6
Knowledge Check Questions.....	7

List of figures

Figure 1: Basic Hello World Program.....	4
Figure 2: Program with User Input.....	4
Figure 3: Program with Formatted Output.....	5
Figure 4: GCC Installation Commands.....	6
Figure 5: Preprocessing Stage.....	6
Figure 5: Compilation Stage.....	6
Figure 6: Assembly Stage.....	7
Figure 7: Linking Stage.....	7
Return Values and Exit Status.....	7
Figure 8: Program with Return Value.....	7
Figure 9: Checking Program Exit Status.....	7

C Program structure and examples

Basic Hello World Program

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    print("Hello World!");
    return 0;
}
```

Figure 1: Basic Hello World Program

Program with User Input

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 2: Program with User Input

Program with Formatted Output

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", &name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height");
    scanf("%f", &height);

    printf("-- User Info --");
    printf("Name: %s\n", name);
    printf("Age: %d\n", age);
    printf("Height: %f\n", height);

    return 0;
}
```

Figure 3: Program with Formatted Output

GCC Compilation Process

```
42 packages can be upgraded. Run 'apt list --upgradable' to see them.
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ sudo apt install build-essential
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 42
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$
```

Figure 4: GCC Installation Commands

Summary

GCC was installed using the build-essential package and its installation was verified using `gcc --version`.

C Compilation Stage Examples

Stage 1: Preprocessing

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ gcc -E task1.c -o task1.i
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ ls
newfile  newfile.c  newfile.i  newfile.o  newfile.s  task1.c  task1.i  task2.c  task3.c  task3.cy  test  test.c  test.i  test.o  test.s  test1  t
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ cat task1.i | less
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$
```

Figure 5: Preprocessing Stage

Stage 2 : Compilation to Assembly

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ gcc -S task1.i -o task1.s
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ ls
newfile  newfile.i  newfile.s  task1.i  task2.c  task3.cy  test.c  test.o  test1  test2
newfile.c  newfile.o  task1.c  task1.s  task3.c  test  test.i  test.s  test1.c  test2.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ cat task1.s
        .arch armv8-a
        .file "task1.c"
        .text
        .section        .rodata
        .align 3
.LC0:
        .string "Hello World!"
        .text
        .align 2
```

Figure 5: Compilation Stage

Stage 3 : Assembly to Object File

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ gcc -c task1.s -o task1.o
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ ls
newfile      newfile.i  newfile.s  task1.i    task1.s    task3.c    test       test.i    test.s    test1.c    test2.c
newfile.c    newfile.o  task1.c    task1.o    task2.c    task3.cy   test.c     test.o    test1     test2
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ file task1.o
task1.o: ELF 64-bit LSB relocatable, ARM aarch64, version 1 (SYSV), not stripped
```

Figure 6: Assembly Stage

Stage 4 : Linking

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ gcc task1.o -o task1
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ ./task1
Hello World!
```

Figure 7: Linking Stage

Summary

GCC was used to demonstrate the four stages of C compilation: preprocessing, compilation to assembly, assembling into an object file, and linking to create the final executable. (.i, .s, .o, and executable)

Return Values and Exit Status

Understanding Return Values

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ ./task1
Hello World!
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ echo $?
0
```

Figure 8: Program with Return Value

Checking Exit Status

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ gcc task1.c -o task1
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ ./task1
Hello World!
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ echo $?
1
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ nano task1.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ gcc task1.c -o task1
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ ./task1
Hello World!
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38$ echo $?
50
```

Figure 9: Checking Program Exit Status

Summary

Tested different return values and used echo \$? to check the program's exit status. 0 indicates success, while non-zero values (1, 50) indicate an error or non-success exit status.

Knowledge Check Questions

Q1. What is the purpose of the #include directive in C programming? Provide an example of its usage.

A: Includes header files. Eg : #include <stdio.h> allows to use scanf, printf.

Q2. Explain the difference between the four stages of compilation. What file extension is produced at each stage?

A: preprocessing (.i), compilation (.s), assembly (.o), linking (executable file).

Q3. What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

A: 0 means successful program execution, non-zero represents non-success exit status or error.

Q4. Write a C program that:

- Declares an integer variable
- Asks the user to input a value
- Displays the entered value back to the user

A:

```
#include <stdio.h>
```

```
int main(){
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered number: %d", number);
    return 0;
}
```

Q5. What GCC command would you use to:

- 1) Only preprocess a file without compiling?
- 2) Generate assembly code but not assemble it?
- 3) Create an object file without linking?

A: gcc -E hello.c -o hello.i

gcc -S hello.i -o hello.s

gcc -c hello.c -o hello.o

Q6. Explain why C is called a "compiled language" and how this differs from interpreted languages.

A: C code is compiled into machine code before execution, whereas interpreted languages are read line by line.